

Appearance Manifold Contributor Guide

Code structure, data format, limitations, and improvement roadmap for runtime weathering interpolation

Audience

This guide is for contributors who extend, maintain, or optimize the plugin. It connects the Weathering Manager Component workflow to the underlying C++ and Python implementation.

1. System Responsibility

The GammaTon module generates weathered textures in the editor. The AppearanceManifold module treats those weathered states as samples in a 7D appearance space, then supports trajectory generation and runtime interpolation.

The runtime path converts key textures into 7D tensors, projects pixels onto trajectory samples, produces an alpha-driven intermediate tensor, reconstructs transient UTexture2D objects, and applies them to a mesh material.

2. Module and File Structure

File	Responsibility
AppearanceManifold.cpp	Registers the Python script path during Startup and checks numpy/scipy/scikit-learn dependencies.
Blueprint Function Library	Blueprint-facing C++ utilities for texture conversion, binary loading, cache generation, interpolation, texture reconstruction, and material application.
Build Nearest Trajectory Async	Runs expensive nearest trajectory lookup on the ThreadPool and returns the completion event on the GameThread.
MainManager.py	Editor-only orchestrator for trajectory generation.
Utility / KNN / MDS / Trajectory	Builds 7D tensors, KNN graph, geodesic distance/MDS embedding, and trajectory paths.
_Reconstructor.py	Python prototype of the runtime interpolation algorithm.

3. Data Model

Every runtime weathering state is represented as seven float values per pixel.

Channel	Meaning
0, 1, 2	BaseColor R, G, B
3, 4, 5	Specular R, G, B
6	Roughness
Array offset	Pixel i starts at $i * 7$.

4. Binary Format

Current implementation

The runtime trajectory binary is $[T:\text{int32}][\text{float} * T * 7]$. Older notes describing a $[T][N]$ header refer to a broader export format and should not be confused with what `LoadWeatheringBinaryData` currently reads.

Field	Description
T	Number of trajectory samples.
Payload	T 7D samples stored as continuous float32 values.
Load function	<code>LoadWeatheringBinaryData(FilePath, OutT)</code> .
Export function	<code>export_to_unreal_runtime_data</code> in <code>MainManager.py</code> .

5. Runtime Call Chain

- 1 Load trajectory samples and T with `LoadWeatheringBinaryData`.
- 2 Convert key textures to 7D tensors with `CombineTextureSources` or `LoadWeatheringTensorsFromFiles`.
- 3 Cache nearest trajectory indices per pixel with `BuildNearestTrajectoryCacheAsync`.
- 4 Generate an alpha-driven intermediate tensor with `InterpolateWeatheringCached`.
- 5 Create or reuse three transient `UTexture2D` objects with `ReconstructTexturesFromTensor`.
- 6 Assign the textures to Base/Spec/Rough parameters on a MID with `ApplyWeatheringTexturesToMesh`.

6. Interpolation Algorithm

Each pixel in key texture A and key texture B owns a nearest trajectory sample index. Given Alpha, the system computes an intermediate index between those two indices, then applies trajectory-sample deltas from both directions before clamping the final 7D result.

The current C++ implementation uses per-channel unrolling and clamp operations. It is simple and fast, but it does not yet perform gaussian neighborhood blending or continuous interpolation between adjacent

trajectory samples.

7. Performance Characteristics

Stage	Cost / Behavior
BuildNearestTrajectoryCache	$O(\text{PixelCount} * T * 7)$. This is the most expensive step and should be asynchronous.
InterpolateWeatheringCached	$O(\text{PixelCount} * 7)$. Relatively light after nearest-index caches are ready.
ReconstructTexturesFromTensor	Builds CPU-side BGRA arrays and uploads them to the GPU through <code>UpdateTextureRegions</code> .
ApplyWeatheringTexturesToMesh	Creates or reuses a MID and updates texture parameters.

8. Current Limitations

- Nearest search is brute-force, so cache generation cost grows quickly with texture resolution and trajectory length.
- `InterpolateWeatheringCached` rounds the intermediate index, so interpolation is not fully continuous between trajectory samples.
- Runtime textures are transient; asset persistence and network replication require additional design.
- Frequent high-resolution texture uploads can cause frame hitches.
- Editor trajectory generation depends on `PythonScriptPlugin` and bundled Python wheel availability.
- Documentation and code must distinguish `Content/WeatheringResults` from older `Saved` path descriptions and must use the current runtime binary format.

9. Improvement Roadmap

Area	Recommendation
Search optimization	Use KD-tree, quantized LUTs, or trajectory segment acceleration structures.
Interpolation quality	Replace rounded index lookup with sample-to-sample linear interpolation or gaussian neighborhood blending.
GPU path	Move reconstruction or upload work to Compute Shader or Render Target based paths.
Cache management	Add version headers, resolution metadata, channel metadata, and safer ActorName/Index file naming.
Error handling	Return node-level failure reasons and separate logging categories.
Packaging	Separate editor-only Python generation from runtime-only loading more explicitly at the module and macro level.

10. Contributor Checklist

- If the 7D channel order changes, update Python export, C++ load, and texture reconstruction together.
- If the binary format changes, add versioning and consider compatibility with existing files.
- If texture update code changes, verify transient texture reuse, resize behavior, GC, and GameThread requirements.
- If async nodes change, ensure Completed delegates are broadcast on the GameThread.
- If Blueprint Component variable names change, update user docs and existing uasset references.
- Benchmark cache build and runtime update cost at 256, 512, and 1024 resolutions.